Enviroment Lab - Explore strace, ltrace, gdb

```c
#include <stdio.h>
void main(int argc, char* argv[], char* envp[])
{
    int i = 0;
    while (envp[i] != NULL)
    {
        printf("%s\n", envp[i++]);
    }
}
```

To compile: `gcc penv.c -o penv`. To execute: `./penv`.

strace

By using `strace`, we can see which system calls our program is performing during its execution

```
$ strace ./penv
execve("./penv", ["./penv"], 0x7fff8755f910 /* 30 vars */) = 0
...
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48, 848) =
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0O{\f\225\\=\201\327\312\301P\32$\230\266\235"...
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f07ba1da000
mprotect(0x7f07ba202000, 2023424, PROT_NONE) = 0
mmap(0x7f07ba202000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x7f07ba397000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000)
mmap(0x7f07ba3f0000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
mmap(0x7f07ba3f6000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1,
close(3)                                = 0
...
write(1, "HOME=/home/utente\n", 18HOME=/home/utente)     = 18
...
exit_group(0)                           = ?
```

ltrace

By using `ltrace`, we can verify which shared functions calls our program is calling

```
$ ltrace ./penv
...
puts("HOME=/home/utente") = 18
...
```

Our program only calls `puts`, which is exported by `libc.so`

gdb lab

1. Compile again the program with `gcc -g penv.c -o penv`. The `-g` argument tells gcc to produce debug informations
2. Debug the program with `gdb ./penv`
3. Use the gdb command `layout src` to split the gdb view in source code + command view
4. Place a breakpoint on main using `b main` and run the program with `r`
5. Execution will pause at the beginning of the main method (first line code of main function is now highlighted)
6. Use the commands `p/a argv` (display argv as a pointer) and `p/a argv` (display envp as a pointer) to obtain their values. They should be near (address may vary in your tests)

> p = print content of variable/memory location/register; a = display pointers

```
(gdb) p/a argv
$3 = 0x7fffffffe068
(gdb) p/a envp
$4 = 0x7fffffffe078
```

> We can see that `argv` is a block of memory starting at `0x7fffffffe068`, and `envp` start at `0x7fffffffe078`, 16 bytes after `argv` (2 pointers in a x64 system)

7. Explore the first two pointers of the `argv` and `envp` list using the commands `x/2a argv` and `x/2a envp` (We expect to see a NULL pointer as the secont)

> x = shows you the contents of a memory address after a dereference, 2 = numbers of items to display; a = display pointers

```
(gdb) x/2a argv
0x7fffffffe068: 0x7fffffffe2f6  0x0
(gdb) x/2a envp
0x7fffffffe078: 0x7fffffffe30b  0x7fffffffe31b
```

Other ways to display these values:

```
(gdb) p/a argv[0]
$5 = 0x7fffffffe2f6
(gdb) p/a argv[1]
```

```
$6 = 0x0
(gdb) p/a envp[0]
$7 = 0x7fffffffe30b
(gdb) p/a envp[1]
$8 = 0x7fffffffe31b
```

8. The pointers inside `argv` and `envp` are our string

```
(gdb) p/s argv[0]
$9 = 0x7fffffffe2f6 "/home/utente/sp/penv"
(gdb) p/s envp[0]
$10 = 0x7fffffffe30b "SHELL=/bin/bash"
(gdb) p/s envp[1]
$11 = 0x7fffffffe31b "WSL2_GUI_APPS_ENABLED=1"
```

9. If we print 3 items starting from `argv`, we will se the "overflow" into `envp`

```
(gdb) x/3a argv
0x7fffffffe068: 0x7fffffffe2f6  0x0
0x7fffffffe078: 0x7fffffffe30b
```